
TEMA 4

Evaluación de Sistemas Informáticos. Monitorización

PROBLEMA 4.1 En un sistema Linux se ha ejecutado la orden `uptime` tres veces en momentos diferentes. El resultado, de forma resumida, es el siguiente:

```
... load average: 6.85, 7.37, 7.83
... load average: 8.50, 10.93, 8.61
... load average: 37.34, 9.47, 3.30
```

Indique si la carga crece, decrece, se mantiene estacionaria o bien no puede decidir sobre ello.

SOLUCIÓN: No se puede decidir sobre la evolución porque no hay una tendencia clara en los valores de las medidas.



PROBLEMA 4.2 En un sistema Linux se ha ejecutado la siguiente orden:

```
$ time quicksort
real 0m40.2s
user 0m17.1s
sys 0m3.2s
```

Indique si el sistema está soportando mucha o poca carga. Razone la respuesta.

SOLUCIÓN: Se trata de un sistema con una carga considerable, ya que hay una notable diferencia entre el tiempo de respuesta que experimenta el usuario (parámetro `real`) y el tiempo que realmente tarda en ejecutarse su programa (suma de los parámetros `user` y `sys`).

PROBLEMA 4.3 Considere las órdenes siguientes ejecutadas en un sistema Linux:

```
$ time simulador_original
real 0m24.2s
user 0m15.1s
sys 0m1.6s
```

```
$ time simulador_mejorado
real 0m32.8s
user 0m10.7s
sys 0m2.1s
```

1. ¿Cuál es el tiempo de ejecución de ambos simuladores?
2. Calcule, si es el caso, la mejora en el tiempo de ejecución del simulador mejorado respecto del original.

SOLUCIÓN: El simulador original se ejecuta en 16,7 segundos y el mejorado en 12,8 segundos (en realidad, los datos muestran que al sistema le cuesta más tiempo porque ha de atender otras tareas). El simulador mejorado es 1,3 veces más rápido que el original.

PROBLEMA 4.4 Se sabe que la sobrecarga (*overhead*) de un monitor software sobre un computador es del 4 %. Si el monitor se activa cada 2 segundos, ¿cuánto tiempo tarda el monitor en ejecutarse por cada activación?

SOLUCIÓN: El monitor tarda en ejecutarse 80 milisegundos por activación.

PROBLEMA 4.5 El monitor *sar* (*system activity reporter*) de un computador se activa cada 15 minutos y tarda 750 ms en ejecutarse por cada activación. Se pide:

Calcular la sobrecarga que genera este monitor sobre el sistema informático.

Si la información generada en cada activación ocupa 8192 bytes, ¿cuántos ficheros históricos del tipo *saDD* se pueden almacenar en el directorio */var/log/sa* si se dispone únicamente de 200 MB de capacidad libre?

SOLUCIÓN: La sobrecarga es del 0,083 % y se pueden almacenar en total 266 ficheros históricos.

PROBLEMA 4.6 El día 8 de octubre se ha ejecutado la siguiente orden en un sistema Linux:

```
% ls /var/log/sar
```

```
-rw-r--r-- 1 root root 3049952 Oct 6 23:50 sa06
-rw-r--r-- 1 root root 3049952 Oct 7 23:50 sa07
-rw-r--r-- 1 root root 2372320 Oct 8 18:40 sa08
```

¿Cada cuánto tiempo se activa el monitor *sar* instalado en el sistema? ¿Cuánto ocupa el registro de información almacenada cada vez que se activa el monitor?

SOLUCIÓN: El monitor se activa cada 10 minutos (la última activación del día se hace a las 23:50 horas). La información generada en cada activación ocupa aproximadamente 21 KB de capacidad.

■

PROBLEMA 4.7 Indique el resultado que produce la ejecución de las siguientes órdenes sobre un sistema Linux con el monitor sar instalado:

1. sar
2. sar -A
3. sar -u 1 30
4. sar -uB -f /var/log/sa/08
5. sar -d -s 12:30:00 -e 18:15:00 -f /var/log/sa/08
6. sadc
7. sadc 2 4
8. sadc 2 4 fichero

SOLUCIÓN:

1. Utilización del procesador durante el día actual.
2. Toda la información recogida durante el día actual.
3. Utilización actual del procesador: 30 medidas tomadas con un período de un segundo.
4. Utilización del procesador y paginación de la memoria virtual durante el día 8 del mes.
5. Transferencias de disco desde las 12:30 hasta las 18:15 horas del día 8 del mes.
6. Información actual en formato binario recogida en una activación del monitor.
7. Información actual en formato binario recogida en dos activaciones separadas 4 segundos.
8. Información actual en formato binario recogida en dos activaciones separadas 4 segundos y depositadas en un fichero del disco de nombre fichero.

PROBLEMA 4.8 Indíquese una orden (u órdenes) que se podría emplear para monitorizar los aspectos siguientes de la actividad en un computador que trabaja con el sistema operativo Linux:

1. Capacidad de memoria física ocupada por un proceso.
2. Número de cambios de contexto por segundo.
3. Carga media del sistema.
4. Número de interrupciones por segundo.
5. Capacidad libre de la unidad de disco magnético.
6. Usuarios conectados a la máquina.
7. Utilización del procesador en modo usuario.
8. Tiempo que lleva ejecutándose un proceso.
9. Tiempo que tarda un proceso en ejecutarse.

SOLUCIÓN:

1. Órdenes top, ps.
2. Órdenes vmstat, sar.
3. Órdenes uptime, sar.
4. Órdenes vmstat, sar.
5. Orden df.
6. Orden who.
7. Órdenes top, vmstat, sar.
8. Órdenes top, ps.
9. Orden time.

PROBLEMA 4.9 Después de instrumentar un programa con la herramienta gprof el resultado obtenido ha sido el siguiente:

Flat profile:

Each sample counts as 0.01 seconds.

% time	cumulative seconds	self seconds	calls	elf /call	total /call	ame
59.36	27.72	27.72	3	9.24	14.39	reduce
33.08	43.17	15.45	6	2.57	2.57	invierte
7.56	46.70	3.53	2	1.76	1.76	calcula

El grafo de dependencias muestra que `invierte()` es llamado desde el procedimiento `reduce()`.

1. ¿Cuánto tarda en ejecutarse el código propio del procedimiento `reduce()`?
2. ¿Cuál es el procedimiento más lento del programa? ¿Y el más rápido?
3. Si el procedimiento más lento de todos se sustituye por otro tres veces más rápido, ¿cuánto tiempo tardará en ejecutarse el programa?
4. Si el procedimiento `invierte()` se sustituye por una nueva versión cuatro veces más rápida, ¿qué mejora se obtendrá en el tiempo de ejecución?
5. Calcule cuál es la aceleración máxima que se podría conseguir en el tiempo de ejecución mediante la optimización del código del procedimiento `invierte()`.

SOLUCIÓN:

1. El código propio de `reduce()` tarda 9,24 s.
2. El procedimiento más lento es `reduce()` y el más rápido es `calcula()`.
3. El programa se ejecutaría en 28,22 s.
4. El programa se ejecutaría 1,33 veces más rápidamente.
5. La máxima aceleración que se podría conseguir es 1,49.

■

PROBLEMA 4.10 Considere la siguiente secuencia de órdenes en un sistema informático donde está instalado el monitor `sar`:

```
% sadc resultado
% ls -l resultado
-rw-r--r-- 1 usuario grupo 712 2006-01-16 10:55 resultado
```

2. Indique qué contiene el fichero `resultado` y cómo se codifica esta información.
3. ¿Qué orden habría que emplear para visualizar en formato ASCII toda la información capturada por el monitor de actividad en la activación anterior?
4. Si el monitor `sar` está instalado en la máquina para ejecutarse cada tres minutos y disponemos de 50 MB de espacio en el disco duro para almacenar la información de actividad, ¿cuántos ficheros históricos diarios podremos mantener en esta instalación?

SOLUCIÓN:

1. El fichero contiene toda la información, en formato binario, recogida sobre la actividad del sistema por la activación del monitor mediante la orden sadc.
2. La orden es sar -A-f resultado.
3. Se pueden mantener 153 ficheros.

PROBLEMA 4.11 La monitorización de un programa de dibujo en tres dimensiones mediante la herramienta gprof ha proporcionado la siguiente información (por errores en la transmisión hay valores que no están disponibles):

Flat profile:

% time	cumulative seconds	self seconds	calls	self s/call	total s/call	name
xxxxx	xxxxx	15.47	3	5.16	5.16	colorea
xxxxx	xxxxx	1.89	5	0.38	0.38	interpola
xxxxx	xxxxx	1.76	1	1.76	3.65	traza
xxxxx	xxxxx	0.46				main

Call graph:

index	% time	self	children	called	name	
[1]	100.0	0.46	19.12		main	[1]
		15.47	0.00	3/3	colorea	[2]
		1.76	1.89	1/1	traza	[3]

		15.47	0.00	3/3	main	[1]
[2]	79.0	15.47	0.00	3	colorea	[2]

		1.76	1.89	1/1	main	[1]
[3]	18.6	1.76	1.89	1	traza	[3]
		1.89	0.00	5/5	interpola	[4]

		1.89	0.00	5/5	traza	[3]
[4]	9.7	1.89	0.00	5	interpola	[4]

1. ¿En cuánto tiempo se ejecuta el programa de dibujo?
2. Indique cuánto tiempo tarda en ejecutarse el código propio de main().
3. Establezca la relación de llamadas entre los procedimientos del programa así como el número de veces que se ejecuta cada uno de ellos.
4. Calcule el nuevo tiempo de ejecución del programa si se elimina el código propio de main() y se reduce a la mitad el tiempo de ejecución del código propio del procedimiento traza().
5. Proponga y justifique numéricamente una acción sobre el programa original que no afecte el procedimiento colorear() (ni su código ni el número de veces que es ejecutado) con el fin de conseguir que el programa se ejecute en 10 segundos.

SOLUCIÓN:

1. El programa se ejecuta en 19,58 s.
2. El código propio de main() se ejecuta en 0,46 s.
3. El procedimiento main() llama 3 veces a colorear() y una vez a traza(); a su vez, traza() llama 5 veces a interpola().
4. El tiempo de ejecución sería de 18,24 s.
5. El tiempo de ejecución no se puede reducir a 10 segundos sin afectar el procedimiento colorear() porque su contribución ya sobrepasa este valor.

PROBLEMA 4.12 Un informático desea evaluar el rendimiento de un computador por medio del benchmark SPEC CPU2006. Una vez compilados todos los programas del paquete y lanzado su ejecución monitoriza el sistema con la orden `vmstat 1 5`. El resultado de las medidas de este monitor es el siguiente:

procs		----- memory-----				---swap--		-----io---		---system		-- ----cpu----			
r	b	swpd	free	buff	cache	si	so	bi	bo	in	cs	us	sy	id	wa
0	0	8	14916	92292	833828	0	0	0	3	0	7	3	1	96	0
1	0	8	14916	92292	833828	0	0	0	0	1022	40	100	0	0	0
3	0	8	14916	92292	833828	2	1	16	3	1016	34	99	1	0	0
1	0	8	14916	92292	833828	0	4	0	8	1035	36	98	2	0	0
2	0	8	14916	92292	833828	1	5	4	28	1035	36	99	1	0	0

Indique si, a la vista de los datos anteriores, los resultados obtenidos en la prueba evaluación serán correctos o no. Justifique la respuesta.

SOLUCIÓN: Los resultados serían incorrectos porque el sistema operativo presenta actividad de intercambio con el disco magnético (swapping).

PROBLEMA 4.13 El resultado de la monitorización de una aplicación informática dedicada al análisis de modelos atmosféricos se muestra a continuación (nótese que hay información no disponible):

Flat profile:

%	cumulative	self		self	total	
time	seconds	seconds	calls	s/call	s/call	name
xxxxx	xxxxx	30.16	52	0.58	0.58	nimbo
xxxxx	xxxxx	5.13	2	2.56	2.56	borrasca
xxxxx	xxxxx	3.51	2	1.75	1.75	lluvia
xxxxx	xxxxx	1.76	1	1.76	34.17	nube

1. Indique cuánto tiempo tarda en ejecutarse el programa.
2. Determine el porcentaje del tiempo de ejecución que consume el procedimiento lluvia().
3. ¿Cuál es el procedimiento más lento de todo el programa?
4. ¿Cuánto tiempo tarda en ejecutarse el código propio de borrasca()?
5. Calcule el nuevo tiempo de ejecución del programa si el procedimiento nimbo() se rediseña y mejora 3 veces.
6. Proponga y justifique numéricamente alguna manera de reducir el tiempo de ejecución del programa original hasta los 20 segundos.

SOLUCIÓN:

1. El programa se ejecuta en 40,56 s.
2. El procedimiento lluvia() es responsable del 8,7 % del tiempo de ejecución.
3. El procedimiento más lento del programa es nube().
4. El código propio de borrasca() se ejecuta en 2,56 s.
5. El nuevo tiempo de ejecución sería de 20,45 s.
6. Mejorando 4 veces el tiempo de ejecución del procedimiento nimbo() el tiempo total del programa se reduce hasta 17,94 s.

PROBLEMA 4.14 Después de conectarse a un sistema informático, un usuario ejecuta las dos órdenes siguientes con el resultado que se muestra:

% uptime

9:50am up 173 days, 23:02, 1 user, load average: 0.00, 0.00, 0.00

% time simulador

real 8m0.70s

user 3m5.20s

sys 0m4.01s

1. ¿En qué condición de carga se encuentra el computador (baja, media o alta) en el momento de conexión del usuario?
2. ¿Cuál es el tiempo (en segundos) de ejecución del programa simulador?
3. ¿Encuentra alguna incoherencia en los resultados anteriores? Justifique la respuesta con argumentos sólidos.

SOLUCIÓN:

1. El computador está en una situación de baja carga.
2. El computador ejecuta el simulador en 480,7 segundos, aunque el tiempo de ejecución efectivo del mismo es de 189,21 segundos.
3. En efecto, el hecho de que la carga sea baja (parámetros de load average a cero) está en contradicción con la espera de 291,49 segundos que experimenta la ejecución del simulador.



PROBLEMA 4.1 En un sistema Linux se ha ejecutado la orden uptime tres veces en momentos diferentes. El resultado, de forma resumida, es el siguiente:

... load average: 6.85, 7.37, 7.83 → estable
... load average: 8.50, 10.93, 8.61 → sube y baja
... load average: 37.34, 9.47, 3.30 → muy alta y luego baja

Indique si la carga crece, decrece, se mantiene estacionaria o bien no puede decidir sobre ello.

PROBLEMA 4.2 En un sistema Linux se ha ejecutado la siguiente orden:

```
$ time quicksort
real 0m40.2s
user 0m17.1s
sys 0m3.2s
```

Indique si el sistema está soportando mucha o poca carga. Razone la respuesta.

Tiene mucha carga ya que la suma de user y sys es menor que el tiempo real

PROBLEMA 4.3 Considere las órdenes siguientes ejecutadas en un sistema Linux:

```
$ time simulador_original
real 0m24.2s
user 0m15.1s
sys 0m1.6s
```

```
$ time simulador_mejorado
real 0m32.8s
user 0m10.7s
sys 0m2.1s
```

1. ¿Cuál es el tiempo de ejecución de ambos simuladores?
2. Calcule, si es el caso, la mejora en el tiempo de ejecución del simulador mejorado respecto del original.

1. Original = $15,1 + 1,6 = 16,7 \text{ s}$
Mejorado = $10,7 + 2,1 = 12,8 \text{ s}$

2. $16,7 / 12,8 = 1,3047 \rightarrow 30,47\% \text{ de mejora}$

PROBLEMA 4.4 Se sabe que la sobrecarga (overhead) de un monitor software sobre un computador es del 4 %. Si el monitor se activa cada 2 segundos, ¿cuánto tiempo tarda el monitor en ejecutarse por cada activación?

$$U_{so} / Total = U_{so} / 2 = 0,04 ; U_{so} = 0,08 \text{ s}$$

PROBLEMA 4.5 El monitor *sar* (*system activity reporter*) de un computador se activa cada 15 minutos y tarda 750 ms en ejecutarse por cada activación. Se pide:

Calcular la sobrecarga que genera este monitor sobre el sistema informático.

Si la información generada en cada activación ocupa 8192 bytes, ¿cuántos ficheros históricos del tipo *saDD* se pueden almacenar en el directorio */var/log/sa* si se dispone únicamente de 200 MB de capacidad libre?

$$\text{Sobrecarga} = \text{Uso} / \text{Total} = 750 / 15 \cdot 60 \cdot 1000 = 8,3 \cdot 10^{-4} \text{ ms}$$

$$\text{Cada activación} = 8192 \text{ bytes} / 15 \text{ min}$$

$$8192 \cdot 4 \cdot 24 = 786432 \text{ bytes/día} = 786,432 \text{ Kb/día}$$

$$200 \cdot 2^{10} / 786,432 = 260,417 \text{ días} = 260 \text{ ficheros completos}$$

PROBLEMA 4.6 El día 8 de octubre se ha ejecutado la siguiente orden en un sistema Linux:

% `ls /var/log/sar`

<code>-rw-r--r--</code>	1	root	root	3049952	Oct	6 23:50	sa06
<code>-rw-r--r--</code>	1	root	root	3049952	Oct	7 23:50	sa07
<code>-rw-r--r--</code>	1	root	root	2372320	Oct	8 18:40	sa08

¿Cada cuánto tiempo se activa el monitor *sar* instalado en el sistema? ¿Cuánto ocupa el registro de información almacenada cada vez que se activa el monitor?

La última ejecución fue a las 23:50 por lo que el monitor se ejecuta cada 10 min

El fichero completo diario ocupa 3049952 bytes

$$1 \text{ activación} = \frac{3049952}{6 \cdot 24} = 21180,2 \text{ bytes}$$

PROBLEMA 4.7 Indique el resultado que produce la ejecución de las siguientes órdenes sobre un sistema Linux con el monitor sar instalado:

1. `sar` → Información del fichero histórico de hoy
2. `sar -A` → Toda la información del fichero histórico de hoy
3. `sar -u 1 30` → Realiza 30 monitorizaciones al segundo
4. `sar -uB -f /var/log/sa/08` Muestra el uso de CPU y memorias del fichero `sa/08`
5. `sar -d -s 12:30:00 -e 18:15:00 -f /var/log/sa/08` Muestra las operaciones E/S de 12:30 a 18:15
6. `sadc` Realiza una monitorización del sistema y se almacena en un fichero binario
7. `sadc 2 4` Monitorización 4 veces cada 2 segundos
8. `sadc 2 4` fichero 7 pero el fichero se llama "fichero"

PROBLEMA 4.8 Indíquese una orden (u órdenes) que se podría emplear para monitorizar los aspectos siguientes de la actividad en un computador que trabaja con el sistema operativo Linux:

1. Capacidad de memoria física ocupada por un proceso. `top/ps`
2. Número de cambios de contexto por segundo. `vmstat/sar`
3. Carga media del sistema. `uptime/sar`
4. Número de interrupciones por segundo. `vmstat/sar`
5. Capacidad libre de la unidad de disco magnético. `df`
6. Usuarios conectados a la máquina. `who`
7. Utilización del procesador en modo usuario. `top/vmstat/sar`
8. Tiempo que lleva ejecutándose un proceso. `top/ps`
9. Tiempo que tarda un proceso en ejecutarse. `time`

PROBLEMA 4.9 Después de instrumentar un programa con la herramienta gprof el resultado obtenido ha sido el siguiente:

Flat profile:

Each sample counts as 0.01 seconds.

% time	cumulative seconds	self seconds	calls	elf /call	total /call	ame
59.36	27.72	27.72	3	9.24	14.39	reduce
33.08	43.17	15.45	6	2.57	2.57	invierte
7.56	46.70	3.53	2	1.76	1.76	calcula

El grafo de dependencias muestra que `invierte()` es llamado desde el procedimiento `reduce()`.

1. ¿Cuánto tarda en ejecutarse el código propio del procedimiento `reduce()`?
2. ¿Cuál es el procedimiento más lento del programa? ¿Y el más rápido?
3. Si el procedimiento más lento de todos se sustituye por otro tres veces más rápido, ¿cuánto tiempo tardará en ejecutarse el programa?
4. Si el procedimiento `invierte()` se sustituye por una nueva versión cuatro veces más rápida, ¿qué mejora se obtendrá en el tiempo de ejecución?
5. Calcule cuál es la aceleración máxima que se podría conseguir en el tiempo de ejecución mediante la optimización del código del procedimiento `invierte()`.

1. 14,39 s

2. El más lento reduce, el más rápido calcula con 1,76 s

3. Es reduce con 9,24 s

$$9,24 / 3 = 3,08$$

$$\text{El total será: } 3,08 + 15,45 + 3,53 = 22,06 \text{ s}$$

4. Ley de Amdahl:

$$G = \frac{1}{(1-f) + f/k} = \frac{1}{(1-0,338) + 0,338/4} = 1,3396 \rightarrow 33,96\% \text{ de mejora}$$

$$5. \lim_{k \rightarrow \infty} G = \frac{1}{1-f} = \frac{1}{1-0,338} = 1,51 \rightarrow 51\% \text{ de aceleración}$$

PROBLEMA 4.10 Considere la siguiente secuencia de órdenes en un sistema informático donde está instalado el monitor sar:

% sdc resultado

% ls -l resultado

-rw-r--r-- 1 usuario grupo 712 2006-01-16 10:55 resultado

- Indique qué contiene el fichero resultado y cómo se codifica esta información.
- ¿Qué orden habría que emplear para visualizar en formato ASCII toda la información capturada por el monitor de actividad en la activación anterior?
- Si el monitor sar está instalado en la máquina para ejecutarse cada tres minutos y disponemos de 50 MB de espacio en el disco duro para almacenar la información de actividad, ¿cuántos ficheros históricos diarios podremos mantener en esta instalación?

2. Contiene la monitorización realizada por sdc cuya información es codificada binariamente

3. Para visualizar la información usamos: sar -A -f resultado

4. Activación = 712 bytes

Al día $60/3 = 20$ activaciones / 24 horas = 480 ejecuciones

$712 \cdot 480 = 341760$ bytes / 1024 bytes = 333,75 Kb

$50 \text{ Kb} / 333,75 = 50 \cdot 2^{10} / 333,75 = 153,408 = 153$ ficheros completos

PROBLEMA 4.11 La monitorización de un programa de dibujo en tres dimensiones mediante la herramienta gprof ha proporcionado la siguiente información (por errores en la transmisión hay valores que no están disponibles):

Flat profile:

% time	cumulative seconds	self seconds	calls	self s/call	total s/call	name
xxxxx	xxxxx	15.47	3	5.16	5.16	colorea
xxxxx	xxxxx	1.89	5	0.38	0.38	interpola
xxxxx	xxxxx	1.76	1	1.76	3.65	traza
xxxxx	xxxxx	0.46				main
		<u>19.58</u>				

Call graph:

index	% time	self	children	called	name	
[1]	100.0	0.46	19.12		main	[1]
		15.47	0.00	3/3	colorea	[2]
		1.76	1.89	1/1	traza	[3]

[2]	79.0	15.47	0.00	3/3	main	[1]
		15.47	0.00	3	colorea	[2]

[3]	18.6	1.76	1.89	1/1	main	[1]
		1.76	1.89	1	traza	[3]
		1.89	0.00	5/5	interpola	[4]

[4]	9.7	1.89	0.00	5/5	traza	[3]
		1.89	0.00	5	interpola	[4]

main llama
a colorear y traza

1. ¿En cuánto tiempo se ejecuta el programa de dibujo?
2. Indique cuánto tiempo tarda en ejecutarse el código propio de main().
3. Establezca la relación de llamadas entre los procedimientos del programa así como el número de veces que se ejecuta cada uno de ellos.
4. Calcule el nuevo tiempo de ejecución del programa si se elimina el código propio de main() y se reduce a la mitad el tiempo de ejecución del código propio del procedimiento traza().
5. Proponga y justifique numéricamente una acción sobre el programa original que no afecte el procedimiento colorear() (ni su código ni el número de veces que es ejecutado) con el fin de conseguir que el programa se ejecute en 10 segundos.

1. 19,58 s

2. 0,46 s

3. Según call graph, main llama 3 veces a colorear y 1 a traza.
Traza llama 5 veces a interpola

4. $15,47 + 1,89 + 1,76/2 = 18,24$

5. No se podría

PROBLEMA 4.12 Un informático desea evaluar el rendimiento de un computador por medio del benchmark SPEC CPU2006. Una vez compilados todos los programas del paquete y lanzado su ejecución monitoriza el sistema con la orden vmstat 1 5. El resultado de las medidas de este monitor es el siguiente:

procs		memory-----				---swap--		-----io---		---system		-- ----cpu----			
r	b	swpd	free	buff	cache	si	so	bi	bo	in	cs	us	sy	id	wa
0	0	8	14916	92292	833828	0	0	0	3	0	7	3	1	96	0
1	0	8	14916	92292	833828	0	0	0	0	1022	40	100	0	0	0
3	0	8	14916	92292	833828	2	1	16	3	1016	34	99	1	0	0
1	0	8	14916	92292	833828	0	4	0	8	1035	36	98	2	0	0
2	0	8	14916	92292	833828	1	5	4	28	1035	36	99	1	0	0

Indique si, a la vista de los datos anteriores, los resultados obtenidos en la prueba evaluación serán correctos o no. Justifique la respuesta.

Las resultados no son correctos por el swapping realizado

PROBLEMA 4.13 El resultado de la monitorización de una aplicación informática dedicada al análisis de modelos atmosféricos se muestra a continuación (nótese que hay información no disponible):

Flat profile:

%	cumulative	self		self	total	
time	seconds	seconds	calls	s/call	s/call	name
xxxxx	xxxxx	30.16	52	0.58	0.58	nimbo
xxxxx	xxxxx	5.13	2	2.56	2.56	borrasca
xxxxx	xxxxx	3.51	2	1.75	1.75	lluvia
xxxxx	xxxxx	1.76	1	1.76	34.17	nube
		<u>40.56</u>				

1. Indique cuánto tiempo tarda en ejecutarse el programa.
2. Determine el porcentaje del tiempo de ejecución que consume el procedimiento lluvia().
3. ¿Cuál es el procedimiento más lento de todo el programa?
4. ¿Cuánto tiempo tarda en ejecutarse el código propio de borrasca()?
5. Calcule el nuevo tiempo de ejecución del programa si el procedimiento nimbo() se rediseña y mejora 3 veces.
6. Proponga y justifique numéricamente alguna manera de reducir el tiempo de ejecución del programa original hasta los 20 segundos.

1. 40,56s

2. $3,51/2 = 1,755 \rightarrow 75,5\%$

3. Nimbo

4. 2,56s

$$5. f_{\text{nimbo}} = 30,16 / 40,56 = 0,744$$

$$t_n = ((1-f) + f/k) t_o = ((1-0,744) + 0,744/3) 40,56 = 20,442 \text{ s}$$

6. Reduciendo drásticamente nimbo

PROBLEMA 4.14 Después de conectarse a un sistema informático, un usuario ejecuta las dos órdenes siguientes con el resultado que se muestra:

% uptime

9:50am up 173 days, 23:02, 1 user, load average: 0.00, 0.00, 0.00

% time simulador

real 8m0.70s

user 3m5.20s

sys 0m4.01s

carga

1. ¿En qué condición de carga se encuentra el computador (baja, media o alta) en el momento de conexión del usuario?
2. ¿Cuál es el tiempo (en segundos) de ejecución del programa simulador?
3. ¿Encuentra alguna incoherencia en los resultados anteriores? Justifique la respuesta con argumentos sólidos.

1. Como vemos las 3 cargas estan a 0, carga baja (nula)

2. 3min y 5.2s

3. Como uptime dice que el sistema no posee ninguna carga no tiene sentido que tarde más tiempo en ejecutar. user + sys es muy diferente al resultado real